LECTURE 2

Context engineering & RAG

21 October 2026

16:00–18:00 CET



LECTURE 2

Agenda

01 **Context engineering**
What goes into an agent's context window.

02 **Reference architecture**
The arXiv curator retrieval flow.

03 **Data ingestion**
PDF parsing with `ai_parse_document`.

04 **Chunking & embeddings**
Prepare information for retrieval.

05 **Vector search**
Lakebase pgvector vs AI Search.



Context engineering

What goes into the context window?

Instructions

System prompt, task rules, examples

User request

Question, intent, constraints

Retrieved knowledge

Relevant documents and paper chunks

Tools & outputs

Tool definitions and observations

Conversation

Recent messages and summaries

Memory & state

Durable facts and task progress

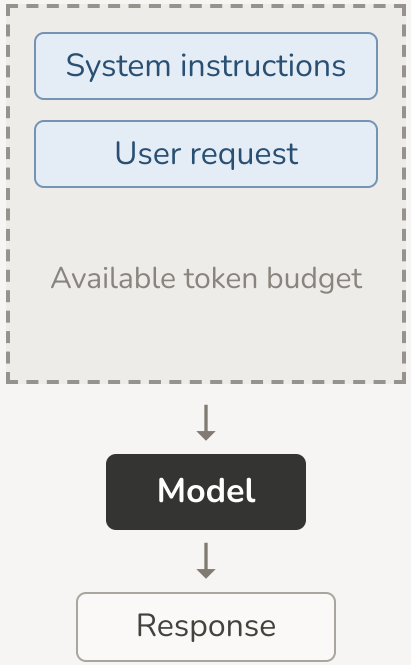
Select and order these components for each model call.



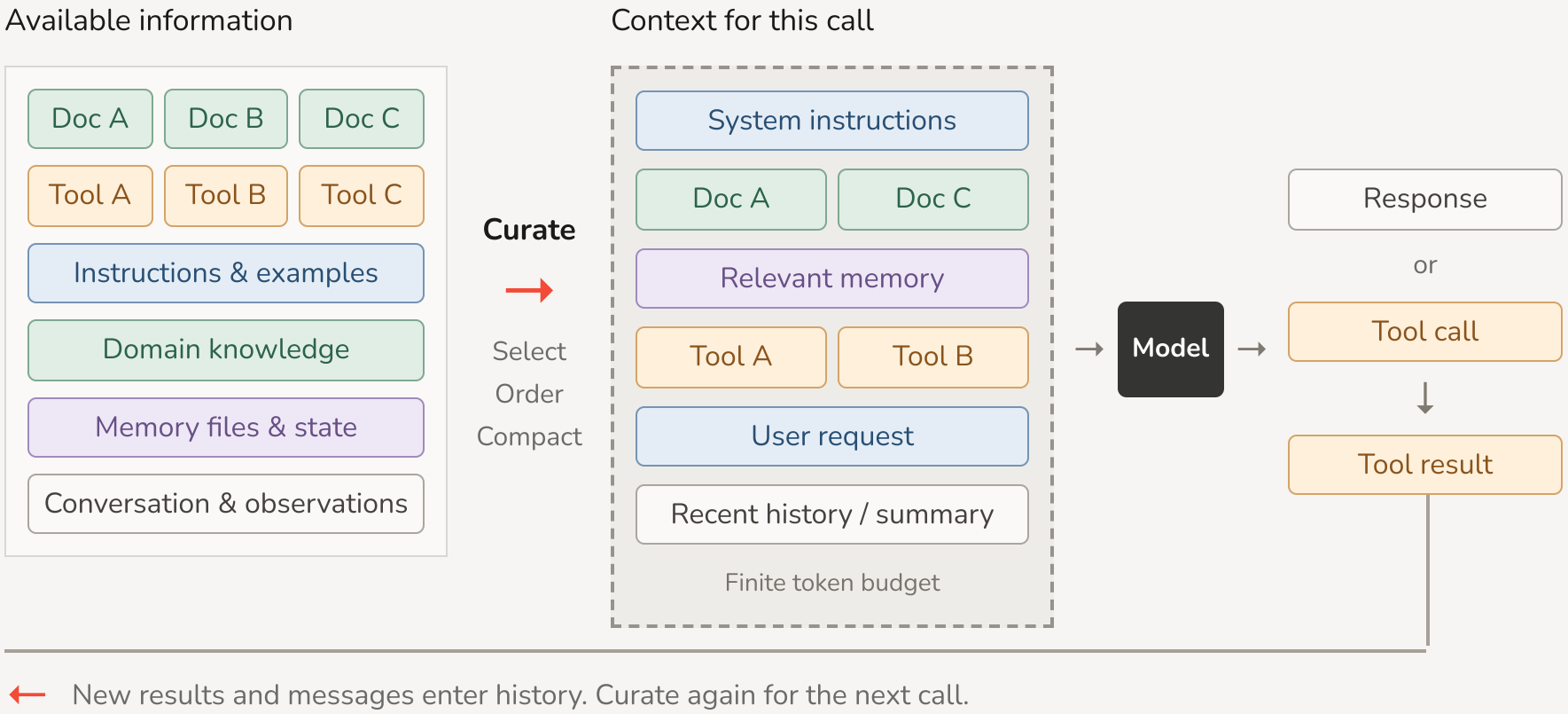
Prompt engineering vs context engineering

Single-turn prompting

Context for one call



Context engineering for agents



Adapted from Anthropic: Effective context engineering for AI agents

The challenge is not “more context”

Prompt engineering

- Instructions and tone
- Static examples
- A carefully written starting point

Context engineering

- External data and retrieval
- Tools, state, and conversation history
- Selection, ordering, and compression at runtime

Every context window is finite.

Useful systems decide what belongs in it, what does not, and how to keep it trustworthy.

Context vs harness vs loop engineering

Layer	Core question	Components
Context	What does the model see on this call?	Retrieval, history pruning, memory, file snippets, state summaries
Harness	How does a run execute safely and reliably?	Sandbox, tool / MCP execution, hooks, guardrails, human approvals
Loop	How does the system repeat, correct itself, and stop?	Reason and act, verification, retries, exit conditions, persistent state



Reference architecture

REFERENCE ARCHITECTURE

Scheduled paper ingestion and search

Scheduled job discovers new arXiv papers

Download

PDFs in a
UC volume

→

Parse

`ai_parse_document`
Structured content

→

Prepare

Chunks + metadata
in Delta

→

Index

Embeddings +
searchable content

---

2 vector search options

Databricks AI Search

Managed vector search index

vs

Lakebase

PostgreSQL + pgvector



PDF parsing challenges

- **Reading order:** headings, columns, and footnotes need structure.
- **Mixed content:** tables, figures, and equations need more than plain text extraction.
- **Text cleanup:** remove repeated headers and repair broken words.
- **Metadata:** retain title, authors, page numbers, and source.
- **Quality:** inspect parsed elements before creating chunks.



arXiv:2610.26845v1 [cs.AI] 19 Oct 2026

TRAIN AN IMAGINATIVE AGENT

Jiaxin Ge[1,*] Junyi Zhang[1,*] Tianning Chai[1,*] Hanjun Yoo[1]
Chenyang Wang[2] Xiuyu Li[2] Weijia Shi[3] XuDong Wang[1,4]
Sewon Min[1] Luke Zettlemoyer[3] Trevor Darrell[1,†] Haiwen Feng[1,2,†]
[1]University of California, Berkeley [2]Impossible Research
[3]University of Washington [4]Duke University

ABSTRACT

A seemingly reasonable push in Sokoban can make the puzzle unsolvable. To avoid such mistakes, an agent has to anticipate the consequences of its actions before execution, especially when feedback from the real environment is sparse or actions are irreversible. We introduce Agentic Imagination (Agima), a training framework that trains an agent to imagine before acting, i.e. to be imaginative. Agima trains a single omni model to propose actions, generate images of their outcomes, and then reason over these imagined futures to decide what to execute in the real environment. Rather than committing to a single plan, the agent can simulate a sequence of actions, explore alternatives, and compare possible futures. We first use supervised fine-tuning (SFT) to unify policy and world modeling within a single omni model. This enables the agent to predict future outcomes and perform basic forms of imagination, such as forward thinking and counterfactual branching. We then use agentic reinforcement learning (RL) to teach the agent to reason with its imagined futures and use them effectively for planning. Across four environments spanning Maze2D, Sokoban, Pacman, and PushT, Agima generalizes to harder settings unseen during training. Trained with SFT alone on two-box Sokoban, Agima solves 40% of four-box puzzles while seeing the real board only once every five moves, compared with an action-only planner that only solves 2%. After RL, new behaviors emerge in how Agima uses its imagination: it learns a new unified format that was unseen during the SFT stage, learns to conduct forward thinking only when necessary, and branches more frequently at critical decision points. An agent that can picture where its actions lead can learn when and where to look ahead, that is, to plan. Project page: https://agima.net/.

1 INTRODUCTION

Agents take actions and receive observations from the environment (Sutton & Barto, 2018; Yao et al., 2023b), but making good decisions often requires anticipating the consequences of actions several steps ahead. When environment interactions are expensive or actions are irreversible, directly testing alternative actions through execution may not be feasible. For example, in Sokoban, a seemingly reasonable push can block another box and make the puzzle unsolvable. To avoid such failures, agents can predict and evaluate the consequences of possible actions before executing them (Racanière et al., 2017).

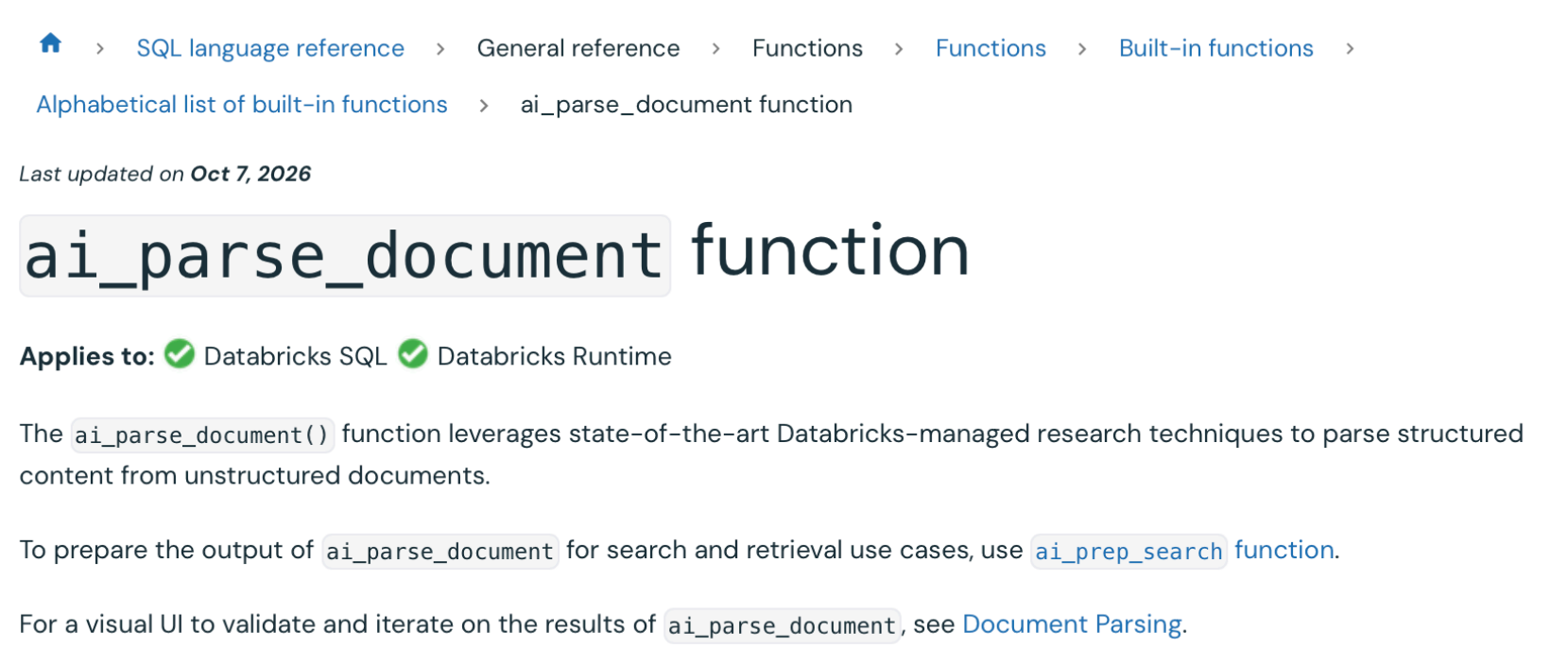
We introduce *Agentic Imagination* (Agima), a training framework that teaches an agent to imagine future observations of its actions and reason over them to plan before acting in the real environment. We build Agima on an omni model that unifies visual generation and language reasoning within the same multimodal context (Zhou et al., 2025; Deng et al., 2025). The same model serves as both the policy model that proposes actions and as the world model that imagines their visual states. These predictions become observations that the agent can use for further planning: it can extend a rollout, explore counterfactual actions, or compare longer candidate plans before execution (Figure 1). Importantly, imagined and real observations share the same interface and can be used interchangeably as inputs to the model. This allows the agent to use imagined observations in place of real

*, †: Equal contribution.

1

Example paper: Train an Imaginative Agent

Structured extraction with ai_parse_document



SQL language reference › General reference › Functions › Functions › Built-in functions › Alphabetical list of built-in functions › ai_parse_document function

Last updated on ***Oct 7, 2026***

`ai_parse_document` function

Applies to: Databricks SQL Databricks Runtime

The `ai_parse_document()` function leverages state-of-the-art Databricks-managed research techniques to parse structured content from unstructured documents.

To prepare the output of `ai_parse_document` for search and retrieval use cases, use `ai_prep_search` function.

For a visual UI to validate and iterate on the results of `ai_parse_document`, see Document Parsing.

Available in SQL, the UI, and via REST API

`POST /api/2.0/ai-functions/ai-parse-document`

Pass a document path in a Unity Catalog volume.

AI FUNCTIONS

ai_parse_document is one of many AI functions

`ai_analyze_sentiment`	Identify sentiment in text
`ai_classify`	Assign your chosen text labels
`ai_decide`	Evaluate questions using criteria or scores
`ai_detect_anomalies`	Find unusual time-series values
`ai_enrich`	Generate fields, optionally using sources
`ai_extract`	Extract fields using a schema
`ai_fix_grammar`	Correct grammar in text
`ai_forecast`	Predict future time-series values
`ai_gen`	Generate a response to a prompt
`ai_generate_text`	Generate text (deprecated: use ai_query)
`ai_mask`	Mask specified entities in text
`ai_parse_document`	Extract structured document content
`ai_predict_class`	Learn from rows to predict missing classes
`ai_predict_value`	Learn from rows to predict missing values
`ai_prep_search`	Prepare contextual chunks for retrieval
`ai_query`	Call a Model Serving endpoint
`ai_search`	Search knowledge sources and ground answers
`ai_similarity`	Score semantic similarity of two strings
`ai_summarize`	Summarize text
`ai_top_drivers`	Find contributors to a metric change
`ai_transcribe`	Transcribe audio with speaker segments
`ai_translate`	Translate text to a target language

Availability, preview status, and compute requirements vary by function.



AI Parse Document: estimated cost

Directional estimates per 1,000 pages in US East

Page complexity	Representative documents	SRTI DBUs	Price / 1k pages
Low: simple text	Legal contracts, memos	10–15	**$0.70–$1.05**
Low: sparse tables or images	Invoices, ID and verification documents	20–25	**$1.40–$1.75**
Medium: complex tables, figures, captions	Company 10-Ks, bank statements	60–65	**$4.20–$4.55**
High: dense forms or complex diagrams	Engineering diagrams, insurance claims, W-2s, 1040s	85–90	**$5.95–$6.30**

Estimates depend on page count, size, and complexity. Use your workload's mix of pages.
Source: Databricks AI Functions pricing, checked 20 October 2026.

Parsed output: two paths to retrieval

	path	parsed_content
56	dbfs:/Volumes/mlops_dev/arxiv/arxiv_papers/202511191940/2511.14715v1.pdf	{"document":{"elements":[{"bbox":[{"coord":[931,49,938,60],"page_id":0}],"content":"1","description":null,"id":0,"type":"page_header"},{"bbox":[{"coord":[82,102,938,183],"page_id":0}],"content":"FLARE: Adaptive Multi-Dimensional Reputation for Robust Client Reliability in Federated Learning","description":null,"id":1,"type":"title"},{"bbox":[{"coord":[82,199,937,240],"page_id":0}],"content":"Abolfazl Younesi* (Student, IEEE), Leon Kiss* Zahra Najafabadi Samani*, Juan Aznar Poveda*, Thomas\nFahringer* (Member, IEEE)","description":null,"id":2,"type":"text"},{"bbox":[{"coord":[82,315,509,828],"page_id":0}],"content":"Abstract—Federated learning (FL) enables collaborative model training while preserving data privacy. However, it remains vulnerable to malicious clients who compromise model integrity through Byzantine attacks, data poisoning, or adaptive adver- sarial behaviors. Existing defense mechanisms rely on static thresholds and binary classification, failing to adapt to evolving client behaviors in real-world deployments. We propose FLARE, an adaptive reputation-based framework that transforms client reliability assessment from binary decisions to a continuous, multi-dimensional trust evaluation. FLARE integrates: (i) a multi-dimensional reputation score capturing performance con- sistency, statistical anomaly indicators, and temporal behavior, (ii) a self-calibrating adaptive threshold

`ai_prep_search` Beta

Opinionated semantic chunking with document context. Produces text ready for embedding.

Do it yourself

This example stores a JSON string. Parse it, extract elements, then apply your own cleanup and chunking.

The `parsed_content` column in this example is a JSON string. Parse it before extracting document elements.



Chunking
& embeddings



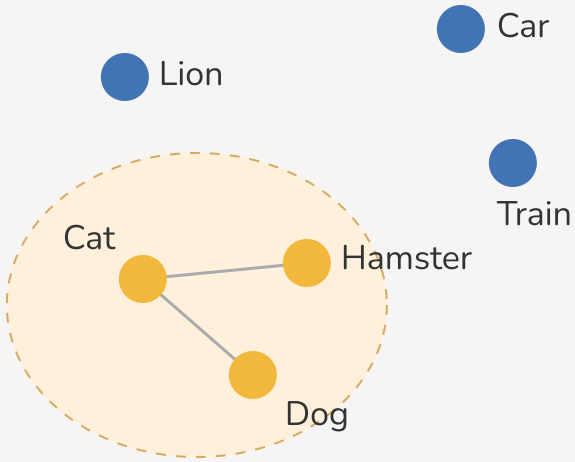
Embeddings: text as semantic vectors

Embeddings represent text as numerical vectors.
Similar meanings produce similar vectors.

"MLOps is awesome" → `[0.1, −0.3, 0.5, …]`

Illustrative vector

Example model	Dimensions	Token window
databricks-gte-large-en	1,024	8,192
databricks-bge-large-en	1,024	512
text-embedding-3-small	1,536	8,192
text-embedding-3-large	3,072	8,192



Illustrative 2D projection

ANN: approximate nearest neighbors.

HNSW: Hierarchical Navigable Small World.
A layered graph that finds nearby vectors quickly, trading some recall for speed.

Tokenization: "MLOps is awesome"

Possible pieces: `[ML] [Ops] [ is] [ awesome]`. The exact split and count depend on the tokenizer. Tokens can be words, word pieces, or punctuation.

Chunking strategy and size trade-offs

Fixed size	Predictable token counts. Overlap helps preserve context at boundaries. **Smaller:** precise retrieval, less context. **Larger:** more context, more noise and tokens.
Sentence / paragraph	Preserves readable units. Length varies, so split units that exceed the token limit.
Semantic / structural	Uses topics, headings, and document elements. Check boundaries and enforce a maximum size.

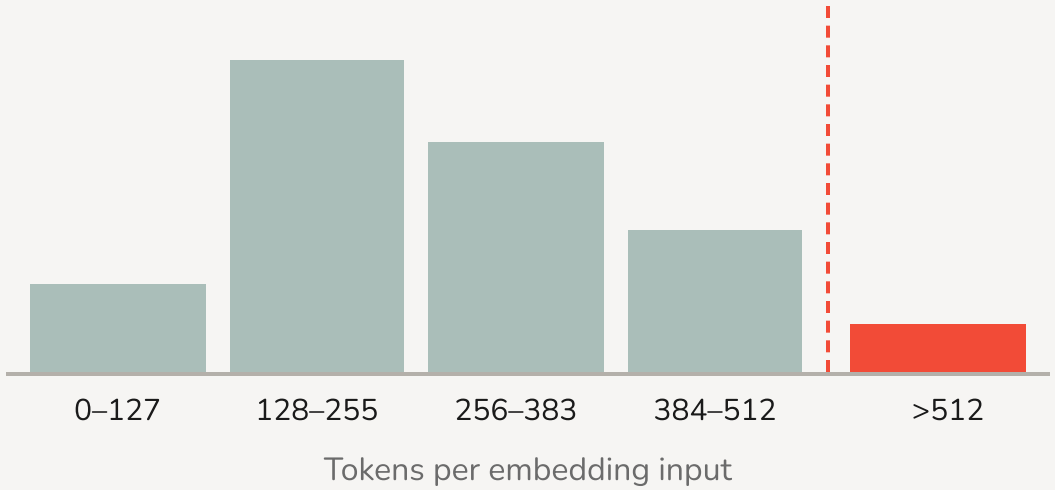
> **Important! Chunk size must fit into the model embedding window!**
>
> `databricks-bge-large-en`: 1,024 dimensions, 512-token window.
> **Maximum chunk size: 512 tokens.** Reserve space for any added title, metadata, or special tokens.



Inspect chunks before indexing

Token-length distribution

Illustrative shape, not course measurements



BGE limit: 512 tokens

Investigate oversized chunks and very short fragments.

Report min, median, p95, max, and count over the limit.

Read a varied sample

Inspect random chunks, the shortest and longest, and samples from different papers.

Check structure and meaning

Look for broken sentences, split tables, missing headings, OCR errors, and repeated headers.

Check records and provenance

Flag empty text and duplicates. Verify chunk IDs, paper IDs, page numbers, and source links.

Check boundaries

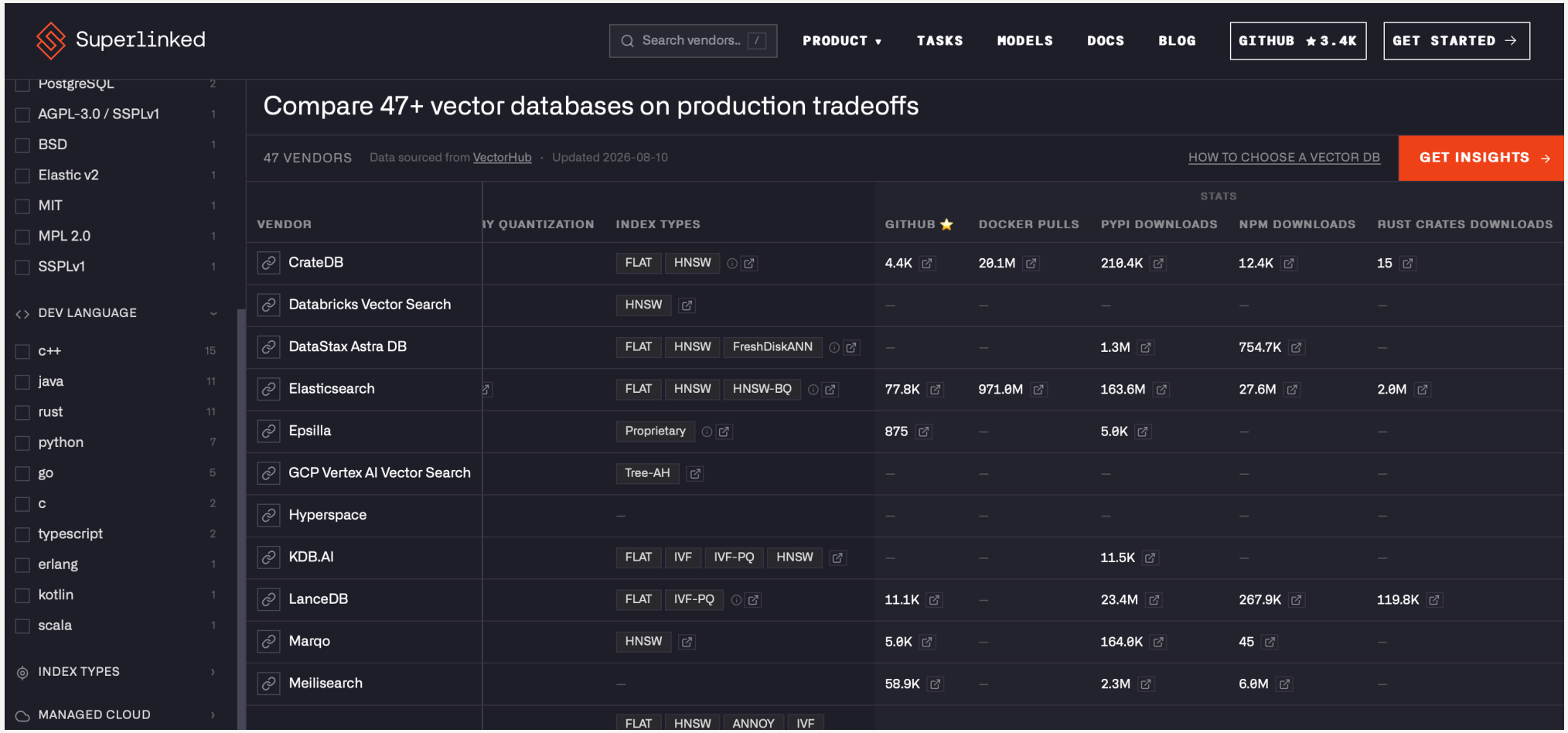
Read neighboring chunks together. Verify overlap and whether each chunk is understandable on its own.

Vector search



VECTOR SEARCH

Compare vector databases



Source: superlinked.com/vector-db-comparison

VECTOR SEARCH

pgvector vs AI Search: search capabilities

Feature	Lakebase + pgvector	Databricks AI Search
Open source	Yes, pgvector extension	✗
License	PostgreSQL license	Proprietary
Filters	SQL WHERE and relational joins	Index metadata filters
Hybrid search	Combine vector + text ranking in SQL	Built-in vector + BM25 rank fusion
Full-text search	PostgreSQL full-text search	Supported; standalone query is Beta
BM25	Additional extension or implementation	Built-in

Lakebase hosts pgvector. PostgreSQL capabilities and additional extensions complement the vector extension.

Superlinked comparison · pgvector · AI Search



VECTOR SEARCH

pgvector vs AI Search: vectors & indexes

Feature	Lakebase + pgvector	Databricks AI Search
Facets	SQL aggregation	✗
Geo search	PostGIS extension	✗
Multi-vector	Multiple columns / indexes; custom scoring	✗
Sparse vectors	sparsevec + HNSW	✗
Vector dimensions	ANN: vector 2,000; halfvec 4,000	4,096 per index
Index types	Exact scan, HNSW, IVFFlat	HNSW

✗ No native support in the referenced comparison. Dimension limits shown apply to ANN indexes.

Superlinked comparison · pgvector · AI Search

ADOPTION

pgvector is a leading choice for vector search

18%

Use among agent developers

Reported using pgvector for agent memory or data management in the 2025 Stack Overflow survey.

3,398 responses to this question. ChromaDB: 20%; Redis: 43%.

Stack Overflow Developer Survey 2025

+24%

Growth in production usage

Datadog identifies pgvector as the fastest-growing non-bundled Postgres extension.

December 2025 to May 2026, within Datadog's observed production databases.

Datadog State of Postgres 2026

Strong adoption and growth support "a leading choice." These samples do not establish overall market leadership or performance superiority.



LAKEBASE EXTENSIONS

Lakebase supports the Postgres ecosystem

Capability	Supported extensions	What they add
Vector search	vector (pgvector)	HNSW and IVFFlat indexes
Geospatial	postgis, pgrouting	Spatial queries and route planning
Text matching	pg_trgm, fuzzystrmatch, unaccent	Similarity, fuzzy matching and normalization
Data structures	hstore, ltree, citext	Key-value, hierarchy and case-insensitive text
Application APIs	pg_graphql, pg_jsonschema	GraphQL and JSON validation
Diagnostics	pg_stat_statements, pgstattuple	Query and storage statistics
Security & IDs	pgcrypto, uuid-ossp	Cryptographic functions and UUIDs

Selected extensions from the official list. Versions depend on PostgreSQL version. Enable pgvector with `CREATE EXTENSION IF NOT EXISTS vector;`

Databricks: complete Lakebase extension list and versions



TWO OPTIONS

AI Search vs Lakebase Search

The preparation is the same: turn PDFs into rows of searchable text.

PDFs in a volume → Parse documents → Table of document chunks

One row per chunk: text + document metadata
Stored as a Delta table in Unity Catalog

Option 1 · AI Search

AI Search generates embeddings from the text column and maintains a search index.

Query with ai_search()

Option 2 · Lakebase Search

Generate embeddings, sync the table to Lakebase, and build a vector index there.

Query with ai_search() or Postgres

We recommend Lakebase for workloads with idle periods. Its compute can scale to zero when enabled; storage remains billable.

ai_search() and both knowledge sources · Lakebase scale to zero



FROM PDF TO SEARCH

Option 1 · AI Search

1 · Parse PDFs

PDFs in a volume → parsed rows

Column	Type
path	STRING
parsed_content	STRING
processed	BIGINT

→

2 · Create chunks

Table of document chunks (Delta)

Column	Type
arxiv_id	STRING
chunk_id	STRING
text	STRING
id	STRING
title	STRING
summary	STRING
authors	STRING
year	INT
month	INT
day	INT
embedding*	ARRAY<FLOAT>

→

3 · AI Search index

Databricks-managed

Select `text` + an embedding model. Databricks embeds chunks and text queries; optionally saves generated vectors.

Bring your own

Generate vectors first. Select the `embedding` column and dimension. Embed queries using the same model.

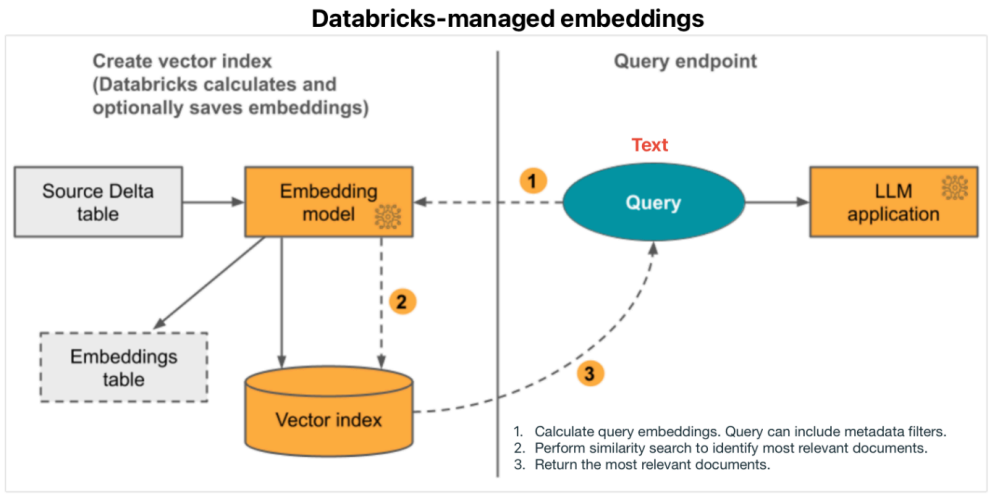
Both: managed SDP sync into the index.

*The embedding column is required for bring-your-own vectors. Choose Triggered or Continuous sync where supported.

Databricks: embedding options

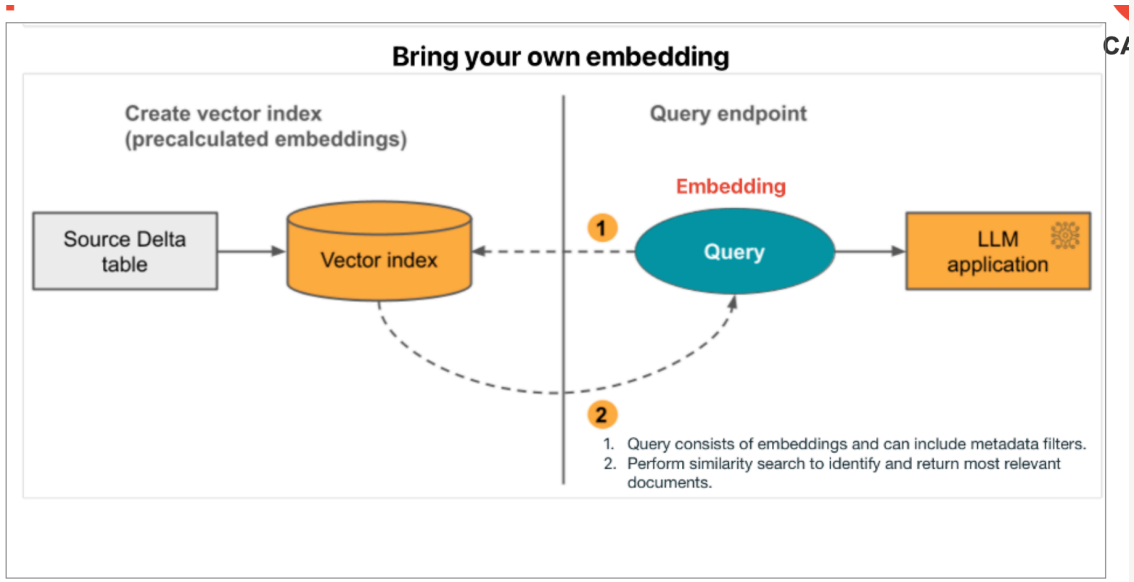
Two ways to provide embeddings

Databricks-managed embeddings



Supply text. Databricks generates document and query embeddings.

Bring your own embeddings



Supply vectors. Use the same model to embed documents and queries.

Diagrams supplied in chat · Databricks: options for providing vector embeddings · Click a diagram to view full size.

Option 2 · Lakebase Search

1 · Parse PDFs

PDFs in a volume → parsed rows

Column	Type
path	STRING
parsed_content	STRING
processed	BIGINT

→

2 · Chunk + embed

Table of document chunks (Delta)

Column	Type
arxiv_id	STRING
chunk_id	STRING
text	STRING
id	STRING
title	STRING
summary	STRING
authors	STRING
year	INT
month	INT
day	INT
embedding	ARRAY<FLOAT>

→

Lakebase synced table

A managed SDP pipeline syncs text, metadata and embeddings into Lakebase.

- Map embedding to `vector(n)`
- Enable Lakebase Search
- Create a `lakebase_ann` index

ai_search()

Or query Postgres directly

Sync: Triggered or Continuous. Use the same embedding model and dimensions for chunks and queries.

Databricks: ai_search() source configuration (Beta)



KEEP SEARCH DATA FRESH

SDP: triggered vs continuous

Spark Declarative Pipelines on Lakeflow (SDP, previously DLT).
Both options use managed SDP pipelines to sync the chunk table to the AI Search index or Lakebase synced table.

Question	Triggered	Continuous
When does it stop?	Finishes the update, then stops	Keeps running until stopped
What does it process?	Data available at the start of each run	New source data as it arrives
Typical freshness needs	Every 10 minutes, hourly or daily	About 10 seconds to a few minutes
Cost / freshness tradeoff	Compute for each run; changes wait for the next trigger	Ongoing compute for lower update lag

Lakebase: choose On-demand → Triggered, or Continuous. Triggered and Continuous require change data feed.
AI Search: standard Delta Sync supports both; storage-optimized endpoints use Triggered.

General SDP guidance: choose the mode based on freshness needs and compute cost. The ranges above are guidelines, not latency guarantees.

SDP mode selection guidelines · Spark Declarative Pipelines on Lakeflow · Lakebase sync · AI Search sync



RETRIEVAL

Search strategies

Semantic search

- Uses embedding similarity
- Finds related meaning and paraphrases
- Can miss exact names, codes or technical terms

```
query_type="ANN"
```

Hybrid search

- Combines vectors with BM25 keyword search
- Preserves exact-term matches alongside meaning
- Combines rankings using reciprocal rank fusion

```
query_type="HYBRID"
```

Start with hybrid, then evaluate.

Compare relevance and latency against semantic search on your own queries.

Databricks: query modes

RETRIEVAL

Querying vector search

Hybrid retrieval with an explicit filter and source fields.

```
# Standard endpoint, Databricks-managed embeddings
results = index.similarity_search(
    query_text="FlashAttention memory-efficient attention",
    query_type="HYBRID",
    columns=["id", "title", "text", "year"],
    filters={"year": 2024},
    num_results=5,
)
```

Compare modes

Use `ANN` for semantic-only retrieval; `FULL_TEXT` for keyword-only search (Beta).

Bring your own vectors

For hybrid search, pass `query_vector` plus `query_text`. Use the same embedding model as the index.

Storage-optimized endpoint: use `filters="year = 2024"`. `index` is an existing AI Search index.

Databricks: SDK queries and filter syntax



RETRIEVAL

Optimizing retrieval quality

First establish an evaluation set, quality metrics and a latency budget.

01 **Enable hybrid search**
Combine meaning with exact terms.

02 **Apply metadata filters**
Constrain retrieval to the intended subset.

03 **Add reranking**
Improve ordering; measure latency and throughput.

04 **Improve data preparation**
Refine parsing, chunking and metadata.

05 **Optimize queries**
Expand or reformulate unclear queries.

06 **Optimize prompts**
Tune extraction and query-rewriting prompts.

07 **Adapt retrieval**
Let agents retry, refine or skip search.

08 **Fine-tune embeddings**
Last resort after diagnosing a model gap.

Reranking trades latency for relevance. Test capacity for high-QPS applications.

Databricks retrieval quality guide: eight steps in order



RETRIEVAL

Metadata filtering

Use structured constraints to narrow the candidate set before ranking.

Date

Limit publication year or time range

Source

Select departments, collections or teams

Type

Separate papers, code and documentation

Access

Apply trusted permission constraints

Language

Choose the corpus language

Category

Restrict topics or document tags

> **“Papers from 2024 about transformers”**
>
> Filter `year = 2024`; use “transformers” as the hybrid search query.

Index the required metadata. Enforce access rules in trusted code; a user-supplied filter is not authorization.

Databricks: metadata filters



RAG pipeline flow

1 **User query**

"What are recent memory-efficient attention methods?"

2 **Query embedding**

Use the document embedding model; managed embeddings handle this step.

3 **Retrieve + rank**

Combine vector similarity, keywords and metadata filters; optionally rerank.

4 **Assemble context**

Select useful chunks, remove duplicates, and retain titles and source IDs.

5 **Generate an answer**

Provide question + evidence to the LLM; request citations and acknowledge missing evidence.



CONTEXT QUALITY

More context can make an answer worse

Too little

- Missed evidence
- Incomplete answers
- Weak source support

Too much

- Noise and conflicting facts
- Higher latency and cost
- “Lost in the middle” attention failures

Goal: the smallest sufficient evidence set.

Retrieve broadly when needed, then select and compress before generation.

Homework



HOMEWORK

Build the retrieval foundation

01 Inspect the processed documents and chunk tables.

02 Review chunk sizes and try one alternative chunking strategy.

03 Create a Lakebase synced table.

04 Run semantic, filtered, and hybrid search; capture what changed.

05 Submit your work through the course pull-request process.



Questions?

Next lecture on MCP & tool calling
& agent architecture

